System Verification and Validation Plan for FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

November 3, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **vi**

2 General Information **1**

2.1 Summary 1

2.2 Objectives 1

2.3 Custom Documents 2

2.4 Relevant Documentation 3

3 Plan **3**

3.1 Verification and Validation Team 4

3.2 SRS Verification 6

3.2.1 Checklist Inspection 7

3.2.2 Supervisor Review 7

3.2.3 Peer and TA Review 7

3.2.4 Traceability Review 7

3.2.5 Continuous Verification 8

3.3 Design Verification 8

3.4 Verification and Validation Plan Verification 9

3.5 Implementation Verification 10

3.6 Automated Testing and Verification Tools 11

3.7 Software Validation 12

4 System Tests **13**

4.1 Tests for Functional Requirements 13
4.1.1 Realistic Flight Simulation (S.2.1) 14
4.1.2 Menu Interface (S.2.2, S.2.5) 15
4.1.3 Checkpoint and Crash Recovery (S.2.3) 16
4.1.4 Timing and Performance Evaluation (S.2.4) 16
4.1.5 Player Accounts and Authentication (S.2.6) 17
4.1.6 Result Storage and Leaderboards (S.2.7) 18
4.1.7 Post-Flight Review and Intelligent Feedback (S.2.8, S.2.9) 19
4.2 Tests for Nonfunctional Requirements 20
4.2.1 Area of Testing1 21
4.2.2 Area of Testing2 21
4.3 Traceability Between Test Cases and Requirements 21

5 Unit Test Description 21

5.1 Unit Testing Scope 23
5.2 Tests for Functional Requirements 23
5.2.1 Module 1 23
5.2.2 Module 2 24
5.3 Tests for Nonfunctional Requirements 24
5.3.1 VR Interface and Input 25
5.3.2 Checkpoint Progress and Recovery 25
5.3.3 Accounts and Online Service 25
5.4 Traceability Between Test Cases and Modules 26

6 Appendix 27

6.1 Symbolic Parameters 27
6.2 Usability Survey Questions 30
6.2.1 Participants and Evaluation Procedure 30
6.2.2 Participant Background 31
6.2.3 Usability Evaluation Questions 32
6.2.4 Additional Feedback Questions 33
6.2.5 Evaluation and Analysis 34

7 Appendix — Generative AI Use Disclosure 35
7.1 AI Tools Used 35
7.2 How and Where Used 35
7.3 Verification of AI-Generated Content 37

8 Appendix — Reflection 40

List of Tables

1 V&V activities by milestone 5
2 Area reviewer rotation 6
3 SRS verification checklist 8
4 Design verification checklist 9
5 V&V plan verification checklist 10
6 Implementation verification checklist 11
7 Required FPVR test configurations 12
8 Traceability between functional requirements and system tests 22

9 Symbolic Constants for System Tests 28

10 Additional Safety and Validation Parameters 29

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Cho et al., 2026b) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

FPVR is a virtual reality (VR)-based first-person-view (FPV) drone training simulator designed to help beginners develop and improve their drone piloting skills in a safe and accessible environment. Instead of requiring a physical drone, users can operate a simulated drone using VR controls and practise essential flight manoeuvres without the risks of equipment damage or unsafe flying conditions. It is also meant as a tool for experienced Drone fliers to be able to practice in a safe and controlled environment without the pressure of piloting a very expensive piece of equipment.

The system features free-flight practice, progressively challenging obstacle courses, multiple virtual environments, and checkpoints to create a structured learning experience. Users can track their performance through metrics such as completion time, checkpoint progress, and performance ratings, with potential AI-driven feedback to help identify areas for improvement. FPVR also supports registered accounts, saved performance records, attempt history, and local and global leaderboards, allowing users to monitor their progress and compare results with others. Overall, FPVR aims to make learning FPV drone piloting more accessible, engaging, and cost-effective through an immersive virtual training experience.

2.2 Objectives

The primary objective of this Verification and Validation plan is to build confidence in the correctness, reliability, usability, performance, and flight realism of FPVR by verifying and demonstrating that all requirements specified in the Software Requirements Specification (SRS) are satisfied. These qualities are major objectives of the SRS and therefore receive explicit attention throughout verification and validation. Verification and validation

efforts will be guided by the Hazard Analysis Document, which will be used to prioritize functionality based on its mission criticality, potential impact, and associated risks. Features that are essential to the safe and reliable operation of the simulator will receive the greatest testing attention to ensure that critical system behaviours are thoroughly evaluated. Due to limitations in project time and resources, validating the long-term effectiveness of VR-based training, and testing compatibility across different VR headsets will remain out of scope. Additionally, testing the correctness of the underlying Unity VR libraries, frameworks, and headset-specific functionality will not be part of the project, as these are external components assumed to function according to their respective specifications. Verification efforts will instead focus on FPVR's implementation and integration of these components.

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

The project will include two Custom Documents: a Usability Report and a Design Thinking Report. The Usability Report will document a systematic and quantitative evaluation of FPVR's usability by having representative users complete predefined tasks within the simulator. The evaluation will establish testing goals, identify specific user tasks, collect feedback through structured questions and surveys, and measure usability outcomes to identify areas for improvement. The Design Thinking Report will document the human-centred design process used to understand the needs and challenges of beginner FPV drone pilots. This will include conducting user interviews, organizing findings through empathy maps, generating and evaluating potential solutions based on feasibility and desirability, and developing two rounds of prototypes to gather user feedback and refine the proposed design. These documents will help ensure that FPVR's design decisions are informed by user needs and that its usability is evaluated through structured user test-

ing.

2.4 Relevant Documentation

This VnV plan is derived from two project documents: the Software Requirements Specification (SRS) (Cho et al., 2026b) and the Hazard Analysis (Cho et al., 2026a). Together they define what FPVR is supposed to do and which failures would be unacceptable, which is the information needed to choose tests and interpret results.

The SRS (Cho et al., 2026b) is the statement of the software requirements. It records the goals of FPVR, the functional requirements (for example flight control, checkpoint behaviour, and session flow), and the non-functional requirements (for example performance, usability, and safety-related qualities). System tests in this plan are written against those requirements so that a passing test set is evidence that the implemented product matches the specified behaviour. The SRS also supplies identifiers, assumptions, and environmental constraints that this plan uses for traceability: each system test should map back to one or more requirements.

The Hazard Analysis (Cho et al., 2026a) identifies hazards and risks for FPVR. Through a Failure Mode and Effect Analysis it describes how components can fail, what the effects of those failures would be, and which safety and security requirements mitigate them. Those hazards and the derived safety requirements focus on the cases where incorrect behaviour could injure a user, damage equipment, produce misleading training, or compromise the integrity of the application. Tests inspired by the Hazard Analysis therefore include failure and mitigation scenarios (for example invalid controller input, lost tracking, or unrealistic physics) in addition to the happy-path cases taken from the SRS.

3 Plan

This section describes how FPVR will be verified and validated. It identifies the V&V team, the plans for verifying the SRS, design, V&V plan, and

implementation, the automated tools that support them, and the plan for validating FPVR with its intended users. Every in-scope requirement will have a planned verification method, traced in Section 4.3 and reported in the V&V Report.

Table 1 links these activities to the course milestones in SRS P.3 and the evidence each one produces.

3.1 Verification and Validation Team

The V&V team follows the project roles in SRS P.1, so each area is verified by the member who knows it best:

- **Ibrahim Tariq:** Team Lead and Flight Physics Developer. Coordinates V&V milestones and defect priorities, and leads flight-model, rendering-performance, and flight-realism testing.
- **John Cho:** Team Liaison and Hardware Integration Developer. Leads headset, controller, installation, and interruption-recovery testing, and arranges supervisor and external reviews.
- **Sankalpa Chhetri Dhakal:** Training Environments Lead. Checks every course, obstacle, and recovery position, and leads the interface usability study.
- **Vikram Chandar:** Game State Developer. Leads tests of game states, checkpoint order, crash events, Respawn, Restart Level, and timers.
- **Arjun Sasidaran:** Scoring and Player Data Developer. Leads tests of scoring, ratings, the post-flight review, accounts, result storage, and leaderboards.
- **Dr. Douglas Down (Supervisor):** Reviews scope, flight realism, and acceptance criteria at the fortnightly meetings, and gives feedback on the Revision 0 demonstration.

Review roles rotate on a fixed schedule so that no one verifies their own work and every member reviews every area during the project. The rotation follows the order Ibrahim, John, Sankalpa, Vikram, Arjun:

Table 1: V&V activities by milestone

Milestone	**V&V activities**	**Evidence**
VnV Plan peer review	Peer and TA review of this plan (Section 3.4)	Logged issues and their resolution
Design Document Rev −1	Review of the MG and MIS, followed by peer review (Section 3.3)	Completed checklist and linked issues
PoC Demonstration	Phases 1–2: deployment, input, safety, and free flight, including the first input-response measurement	Test records and measurement log
Design Documentation	Review of the changed MG and MIS sections (Section 3.3)	Updated checklist and module-to-requirement mapping
Module Implementation	Phase 3: courses, recovery, and timing, including frame-rate and crash-recovery checks; unit and integration suites running in CI	CI test results and coverage report
Revision 0 Demonstration	Phase 4: scoring, review, and local results; supervisor demonstration (Section 3.7)	Demonstration checklist and supervisor feedback
V&V Report	Phase 5: accounts and online results; user studies, mutation exercise, and a full regression run	Study data, test summary, and the V&V Report
Final Demonstration	Phase 6 if included; retest of corrected defects	Retest results

- **Area reviewer:** Each lead keeps ownership of their area's tests, but the member who reviews them changes at each milestone in Table 1, following the rounds in Table 2. After four milestones, every member has reviewed every other member's area, and the rounds repeat.
- **QA reviewer:** The quality assurance (QA) reviewer for each deliverable is the next member in the order. If that member is the deliverable's main author, the turn passes to the following member.
- **Pull request reviewer:** Every pull request is approved by a member other than its author, normally the current area reviewer for that code.

Assignments are recorded on the GitHub issue for each milestone. Members may swap a turn if the swap is recorded and neither member reviews their own work.

Table 2: Area reviewer rotation

Area lead	**Round 1**	**Round 2**	**Round 3**	**Round 4**
Ibrahim	John	Sankalpa	Vikram	Arjun
John	Sankalpa	Vikram	Arjun	Ibrahim
Sankalpa	Vikram	Arjun	Ibrahim	John
Vikram	Arjun	Ibrahim	John	Sankalpa
Arjun	Ibrahim	John	Sankalpa	Vikram

3.2 SRS Verification

The SRS will be verified through five activities: a checklist inspection, a supervisor review, peer and TA review, a traceability review, and continuous verification as the requirements change. Every finding is logged as a GitHub issue with an owner and a closing condition. Table 3 summarizes the activities.

3.2.1 Checklist Inspection

Each area lead will prepare the requirements for their area, and a different member will inspect them using the course SRS checklist. The inspection checks that each requirement is unambiguous, testable, complete, and consistent with the rest of the SRS. Two members will also independently calculate expected results from the scoring, rating, and leaderboard rules; different answers show that a rule is ambiguous.

3.2.2 Supervisor Review

A structured review will be held with Dr. Douglas Down at one of the fortnightly meetings. Before the meeting, the team will share a summary of the key requirements, the main training scenarios, and specific questions about high-risk areas. During the meeting, the team will walk through free flight, a course with a crash and Respawn, and a completed result, focusing on flight realism, the reference drone, and whether the score and feedback support beginner practice. Decisions will be recorded in the meeting minutes.

3.2.3 Peer and TA Review

Issues raised by the assigned peer team and feedback from the TA will be checked against the same checklist. Each one will either be accepted with an owner or declined with a recorded reason.

3.2.4 Traceability Review

A non-author will confirm that every requirement traces back to a project goal or hazard and forward to at least one test, and that each safety requirement in the Hazard Analysis is covered by an SRS requirement. Gaps will be logged as issues, and the result is the traceability table in Section 4.3.

3.2.5 Continuous Verification

Requirements will be reviewed again whenever they change, at the weekly team meetings. These reviews will:

- Assess the impact of each requirement change on the design and tests
- Update the affected tests and traceability entries with the requirement
- Confirm at the PoC Demonstration that the performance requirements can be met on the target headset

Table 3: SRS verification checklist

Activity	**Verification tasks**
Checklist inspection	☐ Inspect each area's requirements ☐ Calculate rule examples independently
Supervisor review	☐ Share the review package ☐ Walk through the training scenarios ☐ Record decisions
Peer and TA review	☐ Accept or decline every issue with a reason
Traceability review	☐ Trace requirements to goals, hazards, and tests ☐ Log gaps as issues
Continuous verification	☐ Review requirement changes weekly ☐ Confirm performance requirements at the PoC

3.3 Design Verification

The Module Guide (MG) and Module Interface Specification (MIS) will be verified through walkthroughs, peer review, and supervisor review, first for Design Document Rev −1 (Nov. 16, 2026) and again for the Design Documentation (Jan. 18, 2027). Each area lead will walk a non-author member through their modules' interfaces and state transitions. Classmates will review the design using a checklist during the scheduled peer reviews (Nov. 19,

2026 and Jan. 25, 2027), and their feedback will be tracked in GitHub Issues. The design will use the nine modules of Section 5, so each module's unit tests trace to its requirements.

A review with Dr. Douglas Down will verify three critical aspects of the design: (1) the flight model and its match to the reference drone, (2) the state transitions for checkpoints, crash recovery, and timing, and (3) the handling of lost input and an unavailable online service, which must not block local training. Small prototypes will check uncertain hardware and performance assumptions before the review.

Table 4: Design verification checklist

Focus area	**Verification tasks**
Module structure	☐ Every requirement has an owning module ☐ Modules match the nine modules in Section 5
Interfaces	☐ Inputs, outputs, units, and valid ranges specified ☐ Failure results specified
State transitions	☐ Checkpoint order and crash penalties preserved ☐ Respawn and Restart Level behaviour defined
Failure handling	☐ Lost tracking and input handled ☐ Online failures do not block local training
Testability	☐ Input, time, and services can be controlled in tests

3.4 Verification and Validation Plan Verification

The V&V Plan will be verified through team and peer review. The QA reviewer, the assigned peer team, and the TA will check the plan's coverage and feasibility using a checklist during the VnV Plan peer review (issues due Nov. 5, 2026), and the QA reviewer will repeat the check after major requirement or design changes. Feedback will be tracked through GitHub Issues and addressed at the weekly team meetings.

To confirm that the tests can detect faults, the team will run a mutation

exercise once the automated suites exist. Faults will be introduced one at a time on a temporary branch, such as allowing an out-of-order checkpoint, removing a crash penalty, or changing a rating boundary, and each must cause a test to fail. Progress will be measured by requirement coverage, the share of mutations detected, and the number of open review issues.

Table 5: V&V plan verification checklist

Criteria	**Verification tasks**
Coverage	☐ Every in-scope requirement has a test or inspection ☐ Safety requirements are covered
Test quality	☐ Each test has inputs, an expected result, and a pass criterion ☐ Expected results are derived independently
Feasibility	☐ Equipment, people, and time are available ☐ No procedure is unsafe for participants
Fault detection	☐ Mutation exercise completed
Process	☐ Feedback tracked to closure

3.5 Implementation Verification

Implementation verification will use unit, integration, and system tests. The unit tests in Section 5 check each module's rules in isolation using the Unity Test Framework. Integration tests check the flows between modules, such as input to flight, crash to recovery, and completion to review and storage. The system tests in Section 4 check the complete application on the required hardware configurations in Table 7.

Static verification will use code review and analyzers. Every pull request will be reviewed by a non-author against the Microsoft C# Coding Conventions adopted in the Development Plan, and merged only after approval and a passing CI run. Reviews will focus on:

- Units, bounds, and event ordering in flight and game-state code

- Cleanup of event subscriptions and unintended shared state
- Credential exposure and authorization errors in account and storage code

Performance testing will verify:

- Frame rate on the target headset (NFR-PERF-01)
- Input response time (NFR-PERF-02)
- Reliability over repeated crash and Respawn cycles (NFR-REL-01)

Each failure will become a GitHub issue, and the failing case and affected regression tests will be rerun after the fix.

Table 6: Implementation verification checklist

Category	**Verification tasks**
Functionality	☐ Unit tests for each module ☐ Integration tests for module flows
Quality	☐ Static analysis completed ☐ Code reviews conducted
Performance	☐ Frame-rate measurements ☐ Input-response measurements
Safety	☐ Pause and exit verified ☐ Tracking and input loss handled

3.6 Automated Testing and Verification Tools

Unit Testing Framework: The Unity Test Framework, which is based on NUnit, will run Edit Mode tests for game rules and Play Mode tests for physics and scene behaviour. It was chosen because it is Unity's standard testing tool and runs in the CI pipeline.

Code Coverage Tools and Plan for Summary: The Unity Code Coverage package will report line coverage for each module and overall, excluding third-party and test code. The target is at least 80% line coverage of the game-rule logic, such as checkpoint order, timing, scoring, and leaderboard ranking. Coverage reports will be kept with each CI run to track trends and identify untested code.

Linters and Static Analysis: The C# compiler and Roslyn analyzers will enforce the Microsoft C# Coding Conventions and flag likely defects during development and in the CI build.

Continuous Integration: GitHub Actions will be extended from building the documentation to building the project and running the automated tests on every pull request. A failing run blocks the merge.

Performance Measurement: The Unity Profiler, headset performance metrics, and timestamped logs will measure frame rate and input response on the configurations in Table 7.

Table 7: Required FPVR test configurations

Mode	Execution and display	Flight input
Standalone	FPVR runs on Meta Quest 3S without a connected PC.	Quest motion controllers
PCVR	FPVR runs on the Windows 11 reference PC and displays through Quest 3S.	Xbox-compatible gamepad

3.7 Software Validation

FPVR will be validated with the people it is built for, because requirement-based tests cannot show whether it is a useful trainer. Validation will use:

- **Interface usability study (NF-UI-01):** About four beginners with little or no FPV experience, split between the two configurations, and the experienced pilots below will complete core tasks without coaching:

finding the control guide, starting a course, using Respawn and Restart Level, pausing and exiting, and reading the review. Task completion and questionnaire ratings (Section 6.2) will be recorded, with a target median of at least 4/5. Sankalpa leads this study.

- **Flight realism study (NF-REAL-01):** About two experienced FPV pilots will fly agreed manoeuvres and rate how believable the flight response is, with a target of at least 4/5 on each item. Ibrahim leads this study, and the supervisor helps define the reference drone.
- **Revision 0 demonstration:** At the Revision 0 Demonstration (Feb. 1–11, 2027), Dr. Douglas Down will review whether the functional requirements in phases 1–4 of SRS P.3 work on both configurations. It succeeds if they can all be demonstrated and no open defect prevents safe use or the core training flow.

Studies will use a clear stationary space, an observer, and a build with verified pause and exit, and participants may stop at any time. Findings will become GitHub issues and will feed the Usability Report and Design Thinking Report.

4 System Tests

System tests are tests that are used to verify that FPVR satisfies the functional and non-functional requirements as specified in the SRS. This section will cover all system tests that will be performed end-to-end on the application.

4.1 Tests for Functional Requirements

The following is a list of tests used to cover the functional requirements (S.2) of the SRS. Note that tests are not mutually exclusive to a single subset of requirements and can hit multiple similarly related major functional modules.

4.1.1 Realistic Flight Simulation (S.2.1)

1. **T.FR.FLT.01**
   Control: Manual

   Initial State: System is running and displayed on a flat screen/monitor or VR headset.

   Input: Move the drone using the corresponding input controller.

   Output: The field of view of the drone changes in response to the input of the controller and moves in the expected direction within the environment (FR-FLT-01).

   Test Case Derivation: Verifies FR-FLT-01.

2. **T.FR.FLT.02**
   Control: Automatic

   Initial State: Simulated drone is at rest within an environment.

   Input: Simple flight path that exercises each of the drone's control axes one by one.

   Output: Drone behaves as expected and follows the path within a reasonable margin of error ($\leq 5\%$ of expected path).

   Test Case Derivation: Verifies FR-FLT-02, FR-FLT-03, FR-FLT-05.

   How test will be performed: Starting at rest (FR-FLT-05), the drone begins by moving up and down, tilting left, right, forward, backward, and rotating in place (FR-FLT-02), followed with sustained flight for a period of time (FR-FLT-03).

3. **T.FR.FLT.03**
   Control: Automatic

   Initial State: Drone is suspended in flight within an environment.

   Input: Drone is moving and projected to hit a collision object.

   Output: Drone comes into contact with object, stops moving (FR-FLT-04), and triggers exactly one logged crash event (FR-FLT-06).

   Test Case Derivation: Verifies FR-FLT-04, FR-FLT-06.

4.1.2 Menu Interface (S.2.2, S.2.5)

1. **T.FR.MI.01**
   Control: Manual

   Initial State: UI is running and main menu is displayed.

   Input: Free-flight mode is selected from the menu.

   Output: Free-flight mode is loaded and drone is placed in an area with no courses, obstacles littered throughout the field that the drone can collide with (FR-LVL-02), and no attempt or timer limitations (FR-LVL-01).

   Test Case Derivation: Verifies FR-LVL-01, FR-LVL-02.

2. **T.FR.MI.02**
   Control: Manual

   Initial State: UI is running and main menu is displayed.

   Input: User scrolls through the list of available courses, terrains/environments for selected course, and hits the start button.

   Output: Course is loaded and drone is initialized at the starting position of the course with all stale data cleared (FR-LVL-07).

   Test Case Derivation: Verifies FR-LVL-07. Navigating through the course menu also indirectly verifies FR-LVL-03, FR-LVL-04, FR-LVL-05, FR-LVL-06, FR-UI-01, FR-UI-05.

3. **T.FR.MI.03**
   Control: Manual

   Initial State: System is running and drone is in a course.

   Input: Button is pressed on the controller that brings up the help menu/controller guide.

   Output: Small window pops up on the screen with a list of controls and their corresponding actions (FR-UI-04).

   Test Case Derivation: Verifies FR-UI-04.

4.1.3 Checkpoint and Crash Recovery (S.2.3)

1. **T.FR.CHK.01**
   Control: Automatic

   Initial State: A selectable obstacle-course level is loaded with its course definition available for inspection.

   Input: Pass through checkpoints in order, then attempt to pass through a later checkpoint before the next required checkpoint and finally reach the finish.

   Output: Level will record the number of checkpoints passed up until a checkpoint is skipped from which it will stop counting (FR-CHK-02, FR-CHK-03).

   Test Case Derivation: Verifies FR-CHK-01, FR-CHK-02, FR-CHK-03, FR-CHK-08.

2. **T.FR.CHK.02**
   Control: Automatic

   Initial State: A course attempt has completed zero checkpoints, and a second attempt has completed at least one checkpoint.

   Input: Drone crashes, select respawn. Drone crashes again, select restart level.

   Output: Application displays option to respawn and restart level if checkpoints ≥ 1 (FR-CHK-04).

   Test Case Derivation: Verifies FR-CHK-04, FR-CHK-05, FR-CHK-06, FR-CHK-07.

4.1.4 Timing and Performance Evaluation (S.2.4)

1. **T.FR.PER.01**
   Control: Automatic

   Initial State: A new obstacle-course attempt is ready to start.

   Input: Run through multiple checkpoint segments, crash during one segment, record epoch time to completion.

Output: Total timer measures active flight, retains time from crashed segments, stops during the recovery menu, does not reset on Respawn, and stops at completion. The section timer measures the current segment and resets at checkpoint completion and Respawn. The crash count increases once per crash (FR-PER-01–FR-PER-07, FR-PER-10).

Test Case Derivation: Verifies FR-PER-01, FR-PER-02, FR-PER-03, FR-PER-04, FR-PER-05, FR-PER-06, FR-PER-07, FR-PER-10.

How test will be performed: Check $\sum_{i=1}^{n} \epsilon_{s,i} \approx \epsilon_t$ where $\epsilon_{s,i}$ is the section time for segment i and ϵ_t is the total time.

2. **T.FR.PER.02**
Control: Automatic

Initial State: Drone has finished a course attempt.

Input: Complete attempts with known values of T and C, including the score boundaries 0, 399, 400, 599, 600, 799, 800, and 1000.

Output: The correct score and rating are calculated and displayed upon completion of a course (FR-PER-08, FR-PER-09).

How test will be performed: The score equals $\max(0, 1000 - \lfloor T \rfloor - 100C)$ and the rating is Needs Improvement for 0–399, Bronze for 400–599, Silver for 600–799, and Gold for 800 or more.

Test Case Derivation: Verifies FR-PER-08, FR-PER-09.

4.1.5 Player Accounts and Authentication (S.2.6)

1. **T.FR.ACC.01**
Control: Manual

Initial State: The application has network connectivity and the username is not registered.

Input: Register a new username with a valid password, repeat with the same username, and repeat with passwords shorter than 12 characters.

Output: The valid registration succeeds. Duplicate usernames and short passwords are rejected with explanatory messages (FR-ACC-01, FR-ACC-02).

Test Case Derivation: Verifies FR-ACC-01, FR-ACC-02.

2. **T.FR.ACC.02**
   Control: Manual

   Initial State: A registered account exists.

   Input: Sign in with valid credentials, sign out, attempt sign-in with an invalid username and with an invalid password, and try to submit a result while signed out.

   Output: Valid credentials create a session, sign-out ends it, invalid credentials are rejected without identifying which field was wrong, and unauthenticated practice remains available while global submission requires authentication (FR-ACC-03–FR-ACC-05).

   Test Case Derivation: Verifies FR-ACC-03, FR-ACC-04, FR-ACC-05.

4.1.6 Result Storage and Leaderboards (S.2.7)

1. **T.FR.DAT.01**
   Control: Automatic

   Initial State: A completed course result is available on an installation with no prior result for that level.

   Input: Save results with different scores, times, crash counts, and timestamps, then open the local leaderboard.

   Output: The local record contains the level, timestamp, total time, crash count, score, rating, and local player identifier. The leaderboard shows at least the top ten available results and orders them by descending score, shorter time, then earlier timestamp; local-only and synchronized states are distinguished (FR-DAT-01, FR-DAT-04, FR-DAT-06, FR-DAT-07, FR-DAT-10).

   Test Case Derivation: Verifies FR-DAT-01, FR-DAT-04, FR-DAT-06, FR-DAT-07, FR-DAT-10.

2. **T.FR.DAT.02**
   Control: Automatic

   Initial State: An authenticated player has completed a course.

Input: Submit once with connectivity, repeat while the online service is unavailable, restore connectivity, and retry synchronization. Open both leaderboards while the service is unavailable.

Output: A connected result is associated with the player's account and appears globally. A failed submission remains local and can be synchronized after connectivity returns. Service unavailability is reported while the local leaderboard and training modes remain usable (FR-DAT-02, FR-DAT-03, FR-DAT-05, FR-DAT-08).

Test Case Derivation: Verifies FR-DAT-02, FR-DAT-03, FR-DAT-05, FR-DAT-08.

3. **T.FR.DAT.03**
   Control: Manual

   Initial State: A result has been synchronized for a registered player.

   Input: Sign out, sign in on another supported installation, and open the player's stored results.

   Output: The previously synchronized result is retrievable and shows the level, timestamp, total time, crash count, score, rating, and player identifier (FR-DAT-09, FR-REV-05).

   Test Case Derivation: Verifies FR-DAT-09, FR-REV-05.

4.1.7 Post-Flight Review and Intelligent Feedback (S.2.8, S.2.9)

1. **T.FR.REV.01**
   Control: Manual

   Initial State: The user has completed an obstacle-course level.

   Input: Observe the review, retry the same level, choose another level, and return to the main menu.

   Output: The review appears immediately and shows the level, final completion time, completed checkpoints, crash count, score, and rating. Each navigation choice works without restarting the application (FR-REV-01– FR-REV-03).

   Test Case Derivation: Verifies FR-REV-01, FR-REV-02, FR-REV-03.

2. **T.FR.REV.02**
   Control: Automatic

   Initial State: A course can be completed while the network and intelligent feedback service are independently available or unavailable.

   Input: Complete attempts in each service state and inspect the post-flight review.

   Output: The standard review is always displayed. When intelligent feedback is available, it is shown automatically and its natural-language observation is grounded in the attempt's time, checkpoint, and crash data. Feedback failure or unavailability does not block the standard review (FR-REV-04, FR-AI-01–FR-AI-03).

   Test Case Derivation: Verifies FR-REV-04, FR-AI-01, FR-AI-02, FR-AI-03.

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input/Condition:
   Output/Result:
   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input:
   Output:
   How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

Table 8: Traceability between functional requirements and system tests

System test case ID	Requirements verified
T.FR.FLT.01	FR-FLT-01
T.FR.FLT.02	FR-FLT-02, FR-FLT-03, FR-FLT-05
T.FR.FLT.03	FR-FLT-04, FR-FLT-06
T.FR.MI.01	FR-LVL-01, FR-LVL-02
T.FR.MI.02	FR-LVL-03–07, FR-UI-01, FR-UI-05
T.FR.MI.03	FR-UI-04
T.FR.CHK.01	FR-CHK-01–03, FR-CHK-08
T.FR.CHK.02	FR-CHK-04–07
T.FR.PER.01	FR-PER-01–07, FR-PER-10
T.FR.PER.02	FR-PER-08, FR-PER-09
T.FR.ACC.01	FR-ACC-01, FR-ACC-02
T.FR.ACC.02	FR-ACC-03–05
T.FR.DAT.01	FR-DAT-01, FR-DAT-04, FR-DAT-06–07, FR-DAT-10
T.FR.DAT.02	FR-DAT-02–03, FR-DAT-05, FR-DAT-08
T.FR.DAT.03	FR-DAT-09, FR-REV-05
T.FR.REV.01	FR-REV-01–03
T.FR.REV.02	FR-REV-04, FR-AI-01–03

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

Most nonfunctional requirements depend on the full system or target hardware, so they are tested in section 4.2. For example, rendering performance (NFR-PERF-01) must be measured on the headset. The tests below give

earlier evidence at the module level. The input response test is the one exception to isolation, because NFR-PERF-02 measures the time from an input event to the flight update.

5.3.1 VR Interface and Input

1. `ProcessInput_HundredEvents_MeetsResponseLimit`

   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Active drone in an empty Play Mode test scene on the reference PCVR computer.

   Input: 100 simulated flight input events.

   Expected Output: At least 95% of events are within the NFR-PERF-02 limit. Median, 95th percentile, and maximum response times are recorded in a table.

   Requirement Traceability: Supports NFR-PERF-02 (input response).

5.3.2 Checkpoint Progress and Recovery

1. `RespawnCycle_TwentyCrashes_KeepsProgress`

   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Attempt with two checkpoints complete.

   Input: 20 crash-and-respawn cycles.

   Expected Output: Both checkpoints remain complete, and no exceptions occur.

   Requirement Traceability: Supports NFR-REL-01 (crash-recovery reliability).

5.3.3 Accounts and Online Service

1. `Register_ValidPassword_NoPlaintextStored`

   Type: Nonfunctional, Dynamic, Automatic

Initial State: Empty local storage and a fake online service.

Input: Registration with a known test password.

Expected Output: The password does not appear in any locally saved data.

Requirement Traceability: Supports NFR-SEC-01 (credential protection).

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

John Cho, Ibrahim Tariq, Vikram Chandar, Sankalpa Chhetri Dhakal, and Arjun Sasidaran. Hazard analysis for fpvr. https://github.com/SoftwareMandems/FPVR/blob/main/docs/HazardAnalysis/HazardAnalysis.pdf, 2026a.

John Cho, Ibrahim Tariq, Vikram Chandar, Sankalpa Chhetri Dhakal, and Arjun Sasidaran. Software requirements specification for fpvr. https://github.com/SoftwareMandems/FPVR/blob/main/docs/SRS/SRS.pdf, 2026b.

6 Appendix

6.1 Symbolic Parameters

The system tests defined in Section 4 use symbolic constants to specify test conditions, acceptance thresholds, and expected results. Defining these parameters in one location ensures consistency across verification activities and allows their values to be updated without introducing conflicting test criteria.

The constants in Table 9 are derived from the Software Requirements Specification (SRS) and the Hazard Analysis. The SRS provides the primary acceptance baseline, while differences between the two documents will be reconciled through the SRS verification process described in Section 3.2.

Table 9: Symbolic Constants for System Tests

Constant	Value	Description and Requirements
FPS_MIN	72 FPS	Minimum average rendered frame rate in standalone mode (NFR-PERF-01).
T_FPS	120 s	Duration of each rendering performance test (NFR-PERF-01).
T_INPUT	100 ms	Maximum input-to-state-update time for qualifying events (NFR-PERF-02).
N_INPUT	100	Number of input events evaluated during response testing (NFR-PERF-02).
P_INPUT	95%	Minimum proportion of input events meeting the response-time threshold (NFR-PERF-02).
N_RECOVERY	20	Consecutive crash-and-respawn cycles per required obstacle-course environment (NFR-REL-01).
N_ENV	2	Minimum number of distinct virtual environments (FR-LVL-04).
N_LEVEL	3	Minimum number of selectable obstacle-course levels (FR-LVL-03).
N_CHECKPOINT	2	Minimum number of ordered checkpoints per obstacle course (FR-CHK-01).
S_BASE	1000	Initial score used in the performance calculation (FR-PER-08).
P_CRASH	100	Score penalty for each registered crash (FR-PER-08).
S_GOLD	800	Minimum score for a Gold rating (FR-PER-09).
S_SILVER	600	Minimum score for a Silver rating (FR-PER-09).
S_BRONZE	400	Minimum score for a Bronze rating (FR-PER-09).

The numerical score is defined in FR-PER-08 as

$$S = \max(0, S_{\text{BASE}} - \lfloor T \rfloor - P_{\text{CRASH}} C),$$

where T is the accumulated active flight time in seconds and C is the number of registered crashes. Ratings are assigned using the thresholds in Table 9, with scores below S_BRONZE classified as Needs Improvement.

Additional parameters from the Hazard Analysis and the planned validation studies are provided in Table 10.

Table 10: Additional Safety and Validation Parameters

Constant	Value	Description and Source
T_BREAK	30 min	Continuous-play duration before displaying a break reminder (Hazard Analysis SR21).
F_FLASH	3 per second	Maximum permitted number of flashes within any one-second period (Hazard Analysis SR22).
N_CRASH_SUG	3	Consecutive crashes within the same checkpoint section before suggesting free-flight practice (Hazard Analysis SR24).
N_BEGINNER	4	Target number of participants for the beginner interface study (Section 3.7).
N_PILOT	2	Target number of participants for the experienced-pilot realism study (Section 3.7).
B_TASK_PASS	3 of 4	Proposed minimum number of beginners completing each core task without assistance (Section 3.7).
B_RATING	4/5	Proposed minimum median beginner survey rating for each item (Section 3.7).
P_RATING	4/5	Proposed minimum rating from each experienced pilot for each realism item (Section 3.7).

The validation-study parameters represent proposed evaluation targets rather than acceptance requirements established by the SRS. These targets will be reviewed with the supervisor before participant testing.

The SRS also identifies proposed parameters for takeoff clearance, gravity verification, and performance-failure handling (PR-FLT-01, PR-FLT-02, and SR-04). Their numerical values remain subject to approval and will not be treated as finalized acceptance criteria until they are agreed upon and documented.

The Hazard Analysis and SRS currently specify different frame-rate and input-response thresholds. These differences will be resolved through the review process in Section 3.2, with corresponding updates made to the sys-

tem tests and symbolic parameters.

Once the test cases in Sections 4 and 5 are finalized, this section will be reviewed to ensure every symbolic constant used in the tests is defined and that unused parameters are removed.

6.2 Usability Survey Questions

The usability evaluation aims to determine whether FPVR provides an intuitive, comfortable, and useful introductory drone training experience. The survey will assess the clarity of controller instructions, ease of navigation, perceived control responsiveness, effectiveness of training feedback, and overall VR experience.

The survey supports the evaluation of NFR-USAB-01, NFR-SAFE-01, and NFR-SAFE-02, as well as related functional requirements concerning flight controls, training environments, crash recovery, and post-flight reviews. The results will contribute to the project's Usability Report, identified as a Custom Document in the Problem Statement.

6.2.1 Participants and Evaluation Procedure

Participants will be selected from the three intended user profiles identified in Section G.8 of the SRS: novice FPV pilots, casual gamers or technology enthusiasts, and experienced FPV pilots. The study will aim to include approximately 6–9 participants, with representation from each profile where feasible.

Each participant will receive a brief explanation of the simulator, its limitations, and the evaluation procedure. Participation will be voluntary, and participants may stop the session at any time if they experience discomfort. The headset will be used in a clear, stationary testing area, with a team member supervising the session.

Participants will be asked to complete the following activities:

1. Locate the control guide and identify the throttle, pitch, roll, and yaw

controls.

2. Enter free-flight mode, practise basic drone movement, and return to the main menu.
3. Select an introductory obstacle course, navigate checkpoints, and interact with the crash-recovery options if a crash occurs.
4. Complete an obstacle course where feasible and review the recorded performance information, including completion time, crash count, score, and rating.

The team will observe whether participants complete tasks successfully, request assistance, encounter navigation difficulties, or experience discomfort. These observations will supplement the survey responses.

6.2.2 Participant Background

Before beginning the evaluation, participants will answer the following questions to help interpret their feedback in relation to their experience.

Q1. How much previous experience do you have operating FPV drones?

Options: No experience, Beginner, Intermediate, Experienced.

Q2. How familiar are you with virtual reality headsets and VR applications?

Options: Not familiar, Slightly familiar, Moderately familiar, Very familiar, Extremely familiar.

Q3. Which controller configuration did you use during the evaluation?

Options: Meta Quest motion controllers, Xbox-compatible controller.

The execution mode (standalone VR or PCVR) will also be recorded by the evaluator.

6.2.3 Usability Evaluation Questions

Participants will rate the following statements on a five-point Likert scale:

1 = Strongly Disagree, 2 = Disagree, 3 = Neutral,
4 = Agree, 5 = Strongly Agree

Participants may select *Not Applicable* for features they did not experience.

Controller Interaction and Flight Experience

Q4. The instructions clearly explained how to use the throttle, pitch, roll, and yaw controls.

Q5. The drone responded to my controller inputs without noticeable delay.

Q6. The drone's movement was predictable and easy to understand while practising.

Q7. The first-person VR view made it easy to understand the drone's position and direction of movement.

Navigation and Training Experience

Q8. I could easily navigate the menus and select the training mode or level I wanted.

Q9. The obstacle-course objectives and checkpoint progression were easy to understand.

Q10. The difference between restarting a level and respawning at the last completed checkpoint was clear.

Q11. The free-flight mode provided a useful environment for becoming familiar with the controls before attempting obstacle courses.

Performance Feedback

Q12. The post-flight review clearly communicated my performance through completion time, crashes, score, and rating.

Q13. The performance feedback helped me identify areas where I could improve my next attempt.

VR Comfort and Safety

Q14. I was able to operate the simulator comfortably while remaining seated or standing in one location.

Q15. The pause and exit options were easy to find and use without removing the VR headset.

Q16. The simulator clearly communicated that success in virtual training does not guarantee the ability to safely fly a real drone.

Overall Experience

Q17. Overall, FPVR provided an understandable and useful environment for introductory drone flight practice.

6.2.4 Additional Feedback Questions

The following questions will collect qualitative feedback that may not be captured by the numerical ratings.

Q18. Did you experience any discomfort during the VR session, such as dizziness, nausea, eye strain, or disorientation? If so, please describe it. Participants may choose not to answer.

Q19. Which aspect of the simulator did you find most helpful for learning or practising drone controls?

Q20. What difficulties or confusing interactions did you encounter while using the simulator?

Q21. What changes would most improve your overall training experience?

Q22. For participants with previous FPV drone experience: How did the simulated drone's behaviour compare with your experience operating a physical drone? Were any movements or control responses noticeably unrealistic?

6.2.5 Evaluation and Analysis

The results of the survey will be analyzed using both quantitative and qualitative methods. For each Likert-scale question, the team will calculate the average rating and summarize the distribution of responses. Responses marked *Not Applicable* will be excluded from the corresponding numerical calculations.

Results will be grouped by participants' prior FPV drone experience where the sample size permits. This will help determine whether beginners and experienced pilots encounter different usability problems or have different expectations regarding flight realism and training feedback.

The team will also record task completion, required assistance, observed navigation errors, and reported discomfort. Open-ended responses will be reviewed to identify recurring usability concerns and suggestions for improvement.

The findings will be documented in the Usability Report and used to inform changes to the controller guide, interface navigation, training environments, crash-recovery interactions, and performance feedback. Due to the limited number of participants, the results will be treated as formative usability evidence rather than definitive proof of improved real-world drone piloting ability.

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- Claude Opus 5.5, Anthropic
- GPT-6 Astra, OpenAI

7.2 How and Where Used

1. **How were the AI tools used?**

   Claude Opus 5.5 and GPT-6 Astra were used as drafting, reviewing, and consistency-checking aids. GPT-6 Astra was used mainly to brainstorm section structure, propose candidate tests and survey questions, and rephrase technical writing so it followed the course template fields (control, initial state, input, expected output, derivation, and how the test will be performed). Claude Opus 5.5 was used mainly to turn the team's existing SRS, Hazard Analysis, Development Plan, and Problem Statement into draft paragraphs, to keep requirement identifiers and module names consistent across sections, and to apply the same test layout repeatedly. Both tools were also used to compare this plan against the template and to flag missing subsections, unmatched requirement identifiers, unused template comments, and wording that treated a planned activity as a completed result.

   The tools did not choose the product scope, the two custom documents, the testing priorities, or the out-of-scope items. Those decisions were made by the team. AI-assisted text was written to match them, and unused suggestions were discarded.

2. **Which parts of the document were affected by AI assistance?**

   - **Summary:** GPT-6 Astra produced a first-pass overview of FPVR as the software under test. Claude then rewrote it against the Problem Statement and SRS so the summary named the actual product functions (VR FPV training, free flight, obstacle courses, checkpoints, performance metrics, accounts, and leaderboards) rather than generic simulator features.

- **Objectives:** GPT-6 Astra suggested candidate VnV qualities (correctness, reliability, usability) and a list of possible exclusions. Claude drafted the in-scope objective as verifying FPVR's own implementation against the SRS, with Hazard Analysis used to rank mission-critical behaviour, and wrote the out-of-scope list (long-term training effectiveness, other VR headsets, and Unity or vendor headset software). Those exclusions were then checked against the Development Plan.
- **Custom Documents:** Claude drafted the descriptions of the Usability Report and the Design Thinking Report after the team confirmed those two custom documents from the Problem Statement. The draft states what each report will contain (task-based usability evaluation with surveys; interviews, empathy maps, ideation, and two prototype rounds) and why they belong in this VnV plan rather than as unrelated extras.
- **Relevant Documentation:** Claude drafted the explanation of how the SRS supplies goals, functional and non-functional requirements, and identifiers for traceability, and how the Hazard Analysis supplies failure modes and safety requirements that add invalid-input, tracking-loss, and unrealistic-physics tests. Claude also drafted the BibTeX entries that point at the team's SRS and Hazard Analysis PDFs; the URLs were supplied by the team.
- **Plan sections (team, SRS verification, design verification, VnV-plan verification, implementation verification, automated tools, and software validation):** Both tools were used to check that these subsections still follow the template (roles rather than a name list.
- **System tests:** GPT-6 Astra was used to brainstorm groupings of functional tests (flight, courses, menus, recovery, accounts) and non-functional tests (performance, usability, safety) from the SRS. Claude was used to keep each test in the template format and to attach requirement identifiers. Happy-path cases were taken from the SRS; failure cases (invalid controller input, lost tracking, crash recovery) were taken from the Hazard Analysis.
- **Traceability:** Claude was used to draft or check mappings from test identifiers to SRS requirement identifiers and from unit tests

to the nine logical modules in SRS section S.1, so that a requirement was not left with no planned verification.

- **Unit Test Description:** Claude drafted the introduction, the in-scope and out-of-scope module lists, the black-box / white-box / mocking strategy, and the named automated tests for Flight and Collision, Modes and Levels, Checkpoint Progress and Recovery, Timing and Evaluation, VR Interface and Input, Post-Flight Review, Accounts and Online Service, Results and Leaderboards, and Intelligent Feedback. GPT-6 Astra was used as a second pass to suggest extra edge cases (for example repeated contact producing one crash event, or no crash before takeoff).
- **Usability survey appendix:** GPT-6 Astra helped brainstorm Likert items and background questions. Claude drafted the participant procedure (approximately 6–9 people from the three SRS user profiles), the task script (control guide, free flight, course and crash recovery, post-flight review), the five-point rating items, qualitative follow-ups, and the analysis plan.
- **Wording and consistency review:** Both tools were used throughout to shorten sentences, to keep terminology aligned (requirement identifiers, module names, "system test" versus "unit test," stretch goals versus required tests), and to remove leftover template comments from sections the team had already filled.
- **This disclosure:** Claude Opus 5.5 drafted this appendix from the course template and from the team's record of which sections were AI-assisted. GPT-6 Astra was not used to invent a list of uses that did not occur.

AI-generated suggestions were used as drafts and review notes, not as final text. No figures or diagrams in this document were generated by AI. Citations were not invented by AI.

7.3 Verification of AI-Generated Content

1. **How the team checked AI-generated or AI-assisted material before including it in the document.**

Every AI-assisted change was reviewed by the author of that section before it was committed, and again through pull-request review by another team member before it was merged. Drafts were compared with their sources rather than accepted because they sounded complete: the summary and custom documents against the Problem Statement, objectives and tooling against the Development Plan, system and unit tests against SRS identifiers, and failure and safety tests against the Hazard Analysis. When a tool proposed a new feature, a numeric target, or a change of scope, the team confirmed the decision independently. Several suggestions were rejected for that reason, including treating Unity, OpenXR, or Meta system software as in-scope for FPVR unit tests, treating stretch-goal multiplayer or dedicated transmitters as required tests, and stating headset-wide compatibility testing that the objectives section explicitly excludes. Agreement between Claude and GPT-6 Astra was not treated as verification.

2. **How the team verified factual claims, technical assertions, requirements, calculations, and references.**

    - **Requirements and traceability:** Test cases that name a requirement (for example FR-FLT-03, FR-FLT-05, NFR-USAB-01) were checked against the identifiers defined in the SRS. Tests that name a hazard or safety requirement were checked against the Hazard Analysis. If a draft test cited an identifier that does not exist, or described behaviour that the SRS marks as a stretch goal or likely change, the test was rewritten or marked as applying only if that requirement is in scope.
    - **Technical assertions:** Claims about Unity Test Framework, NUnit, Unity Code Coverage, Edit Mode and Play Mode tests, and the Microsoft C# coding standard were checked against the Development Plan and the vendor documentation for those tools. They are recorded as the team's planned tools, not as evidence that the tests have already been executed.
    - **Calculations and expected outputs:** Expected results in unit tests (for example one crash event from repeated contact, or a decrease in vertical velocity under zero throttle) were derived from the corresponding SRS rules and checked by a team member.

Where the flight model is still under review, tests state the direction of change rather than a fabricated numeric trajectory. Score, rating, and timing oracles will use the formulae in the SRS; they are not taken from an AI-produced example.

- **Usability procedure:** Participant counts, user profiles, and task scripts were checked against Section G.8 of the SRS and the Custom Documents section of this plan so that the survey cannot silently invent a different study. Likert items were reviewed to ensure they measure the intended qualities (control clarity, navigation, comfort, feedback) rather than implementation details the participant cannot observe.
- **References:** AI tools were not used to invent citations. Bibliographic entries for the SRS and Hazard Analysis were written by the team and point at the project repository. The document was compiled with `pdflatex` after substantive edits, and the log was checked for undefined citations and missing labels.
- **Plans versus results:** AI-assisted wording was edited so that this document remains a plan. Coverage targets, frame-rate or latency numbers, and usability scores are treated as quantities to be measured later and reported in the VnV Report, not as verified outcomes of the present draft.

3. **Who is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document.**

   The team is responsible for the accuracy, completeness, consistency, and appropriateness of this Verification and Validation Plan, whether a passage was written by a team member or drafted with AI assistance. Claude Opus 5.5 and GPT-6 Astra can miss requirements, invent identifiers, or describe tests the team cannot run with the time and hardware available. Human review against the SRS, Hazard Analysis, Development Plan, and course template is therefore the check that the plan is feasible, internally consistent, and appropriate for FPVR.

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Ibrahim Tariq

1. **What went well?** Using the SRS and Hazard Analysis gave us a clear starting point for choosing tests. Dividing responsibilities helped me focus on flight behaviour and performance while connecting my work to the broader plan.

2. **Pain points and resolutions.** The main challenge was making flight realism and VR comfort testable. I addressed this by separating measurable physics and performance checks from user feedback. Conflicting acceptance thresholds still need agreement before those tests can establish a pass.

3. **Knowledge and skills needed.** I need stronger skills with the Unity Profiler to identify frame-time bottlenecks and measure input responsiveness on our target hardware. I also need practice designing repeatable physics tests for gravity, controller inputs, and collisions, including choosing meaningful tolerances so I can distinguish implementation errors from limitations of the flight model.

4. **Learning approaches and choices.** I can improve my profiling skills by studying Unity's documentation or analysing captures from controlled flight scenes. I will focus on analysing captures so I can connect frame-time spikes to specific behaviour. For physics testing, I can follow Unity Test Framework tutorials or write small Play Mode tests with fixed inputs. I will prioritise writing tests, using tutorials as references, because this gives me practice isolating variables and reproducing failures.

Sankalpa Chhetri Dhakal

1. **What went well while writing this deliverable?**

   Having our finalized SRS and Hazard Analysis gave us a good foundation for developing the V&V Plan since we already had a clear understanding of the system's requirements and potential risks. I primarily worked on the symbolic parameters and usability survey questions, which helped me think about how we would evaluate FPVR beyond

just checking whether the software works. I also spent time reviewing my teammates' sections, which was useful for identifying missing details and making sure our verification strategies aligned with the requirements. Overall, dividing the document into sections made the workload more manageable.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   One of the main challenges was that different sections of the V&V Plan were closely connected, making it difficult to complete everything independently. For example, my symbolic parameters depended on the system tests written by another teammate, while my usability survey needed to align with the validation procedures described earlier in the document. We also identified inconsistencies between our SRS and Hazard Analysis, particularly regarding performance thresholds and which features were considered core requirements. Rather than ignoring these differences, we documented them for further review and worked on aligning our testing procedures with the finalized SRS.

3. **What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?**

   Our team will need to strengthen several technical and testing skills to properly evaluate FPVR. Ibrahim will need to develop experience validating drone flight physics and measuring VR performance using Unity's profiling tools. John will focus on testing controller mappings, headset tracking, and compatibility between standalone and PCVR configurations. Vikram will need to become familiar with automated unit and integration testing using Unity Test Framework and NUnit, especially for checkpoint progression and crash recovery. Arjun will focus on verifying scoring accuracy, data persistence, and authentication security. I will need to improve my understanding of usability testing, survey design, and user feedback analysis to determine whether our simulator is intuitive and effective for beginners.

4. **For each knowledge area and skill identified above, what are at least two approaches to acquiring the knowledge or mastering**

the skill? Which approaches will each team member pursue, and why?

Each team member can develop these skills through studying technical documentation and tutorials or through practical experimentation with our project. Ibrahim will combine research on drone dynamics with controlled Unity simulations to compare expected and actual flight behaviour. John will use official XR documentation and build small hardware integration prototypes to identify compatibility issues early. Vikram will learn from NUnit tutorials and practise implementing automated tests for game-state functions, while Arjun will study secure data-handling practices and test scenarios involving failed saves and invalid authentication. I plan to review usability testing methods from HCI coursework and conduct structured sessions with potential users. We chose these approaches because combining theory with hands-on testing will help us develop the necessary skills while directly improving FPVR.

John Cho

1. **What went well while writing this deliverable?**

   Having a detailed Software Requirements Specification (SRS) made it much easier to write the system tests for this verification plan. The SRS provided specific requirements, expected behaviours, constraints, and acceptance criteria that could be converted into test objectives and observable pass or fail conditions. Instead of having to decide from scratch what the system tests should cover, I could trace each test back to a requirement and use the requirement's wording to determine the inputs, the expected result, and the evidence that should be collected. This was especially helpful for tests involving controller mappings, headset tracking, installation, and interruption recovery, because the SRS clarified what the user should experience without forcing the test to depend on a particular implementation. The detailed requirements also made it easier to identify missing cases and to check that the system tests covered normal operation as well as important edge cases. I likely would've had a much harder time writing functional tests without a detailed SRS otherwise.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   A major pain point was that several requirements were similar or partially overlapping. As a result, the same behaviour could appear relevant to more than one system test, which made it difficult to determine where each requirement should be referenced and how to link all of the requirements without duplicating tests unnecessarily. It was also easy for a test written for one feature to implicitly cover another feature without making that relationship clear in the traceability table. I resolved this by first categorizing and grouping related requirements, such as input and controller behaviour, headset and tracking behaviour, installation and compatibility, and interruption recovery. I then wrote the system tests category by category, identifying the primary requirements for each test and recording any additional requirements that were also covered. This made the test boundaries clearer and produced more complete and consistent traceability.

3. **What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?**

   The team will need knowledge of Unity and C# development, VR development on the Meta Quest 3S with OpenXR, drone flight dynamics, level and environment design, machine learning (optionally), state-machine design, testing, and generally good game design. These skills are needed to verify that the simulator behaves correctly, provides responsive and usable VR interactions, represents flight realistically, supports coherent training environments, preserves consistent game states, and reports meaningful performance results.

4. **For each knowledge area and skill identified above, what are at least two approaches to acquiring the knowledge or mastering the skill? Which approaches will each team member pursue, and why?**

   The team can acquire these skills through a combination of structured study and hands-on implementation. For Unity and C#, one approach is to use Unity Learn tutorials and the Unity manual, while a second approach is to build small prototypes before implementing full features.

For VR development, the team can study Meta, OpenXR, and XR Interaction Toolkit documentation and sample projects, or test early builds on the Quest 3S while measuring tracking, input, frame rate, and latency. For flight dynamics, the team can read about quadcopter dynamics and the reference drone, or compare the simulator with established FPV simulators. Level design can be learned by studying existing FPV courses and by conducting teammate playtests. Machine-learning skills can be developed through model documentation and small experiments on recorded flight data. State-machine testing can be developed by documenting transitions formally and writing unit tests for each transition. Scoring and flight-data skills can be developed by studying scoring systems in existing simulators and by calibrating an initial scoring formula with recorded team flights.

I will primarily pursue the VR documentation and hands-on Quest 3S testing approaches because my responsibilities include headset, controller, installation, and interruption-recovery testing. I will also use small hardware-integration prototypes to find compatibility and recovery issues early. The other members will combine the approaches that best match their responsibilities: Ibrahim will study flight dynamics and compare controlled simulations, Sankalpa will study and playtest course designs, Vikram will document and unit-test state transitions, and Arjun will study existing scoring systems before calibrating the scoring and data-recording implementation with real runs. Combining research with practical experiments will give the team both the background needed to design meaningful tests and direct evidence that the implementation satisfies the requirements.